



Fully Connected Feedforward Neural Networks

Dr. Sunil Kumar Vuppala

AI Expert, ACM Distinguished Speaker

www.linkedin.com/in/sunilvuppala/



Contents

- Introduction to Deep Learning
- History of Deep Learning
- Neural Network vs Deep Learning
- Structure of a Neural Network
- Tensorflow play ground
- How Deep Learning works?
 - Feedforward in NN
 - Backpropagation in Neural Networks – Network simulator
- Overfitting
- Hyperparameter Finetuning

Let us start with applications of DL

- Finance domain (**Categorical and Numerical** data) :
 - Identify the fraud detection in credit card transactions
- Healthcare domain (**Image** data):
 - Lung cancer classification of images / TB detection from chest images
- Social media(**Image** data):
 - Face recognition and tag the people
- Across the domains (**Text**):
 - Identify the potential cases of automation from historical ticket data
 - Build a chat bot

ARTIFICIAL INTELLIGENCE

IS NOT NEW

ARTIFICIAL INTELLIGENCE

Any technique which enables computers to mimic human behavior



MACHINE LEARNING

AI techniques that give computers the ability to learn without being explicitly programmed to do so



DEEP LEARNING

A subset of ML which make the computation of multi-layer neural networks feasible



1950's

1960's

1970's

1980's

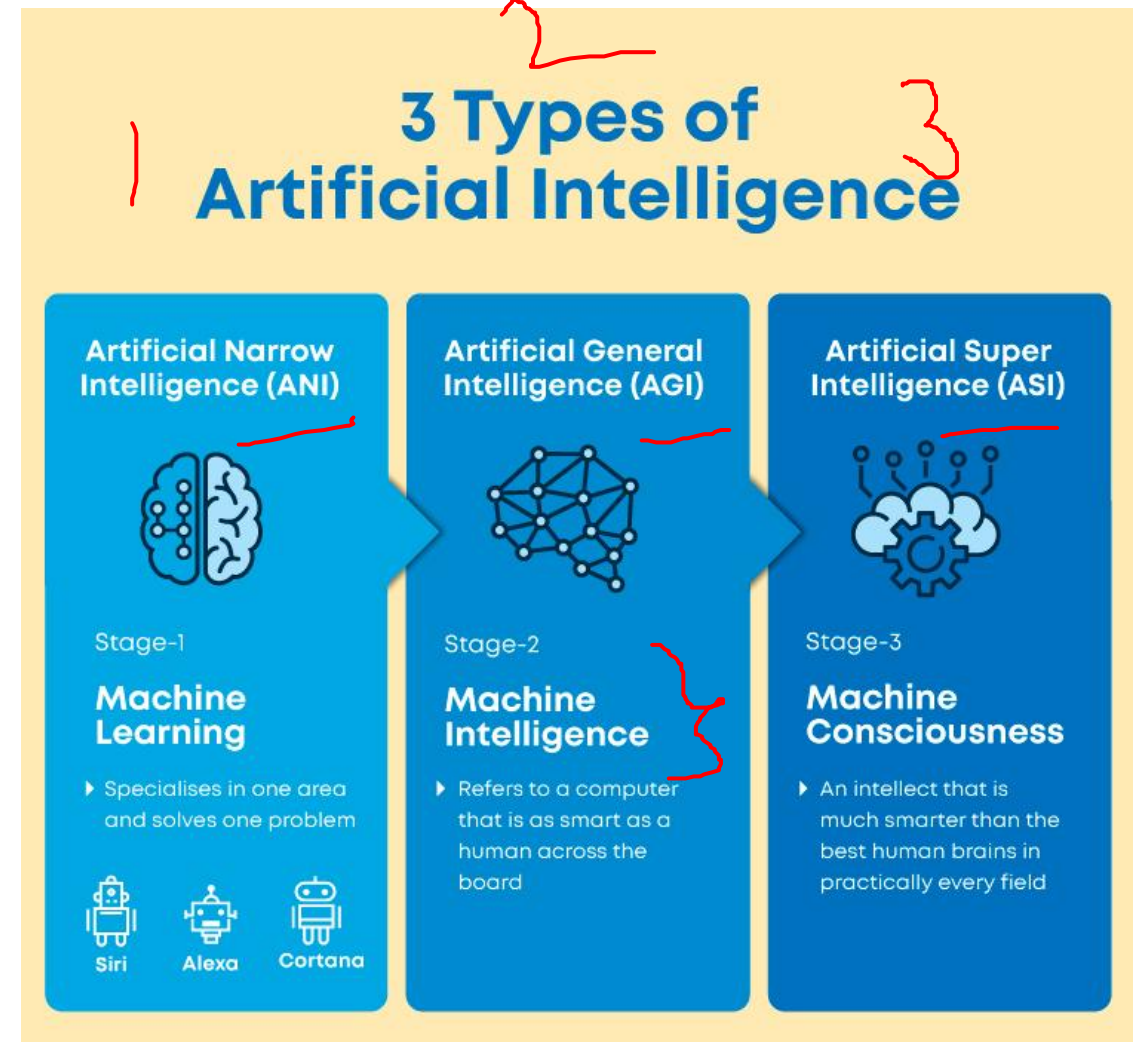
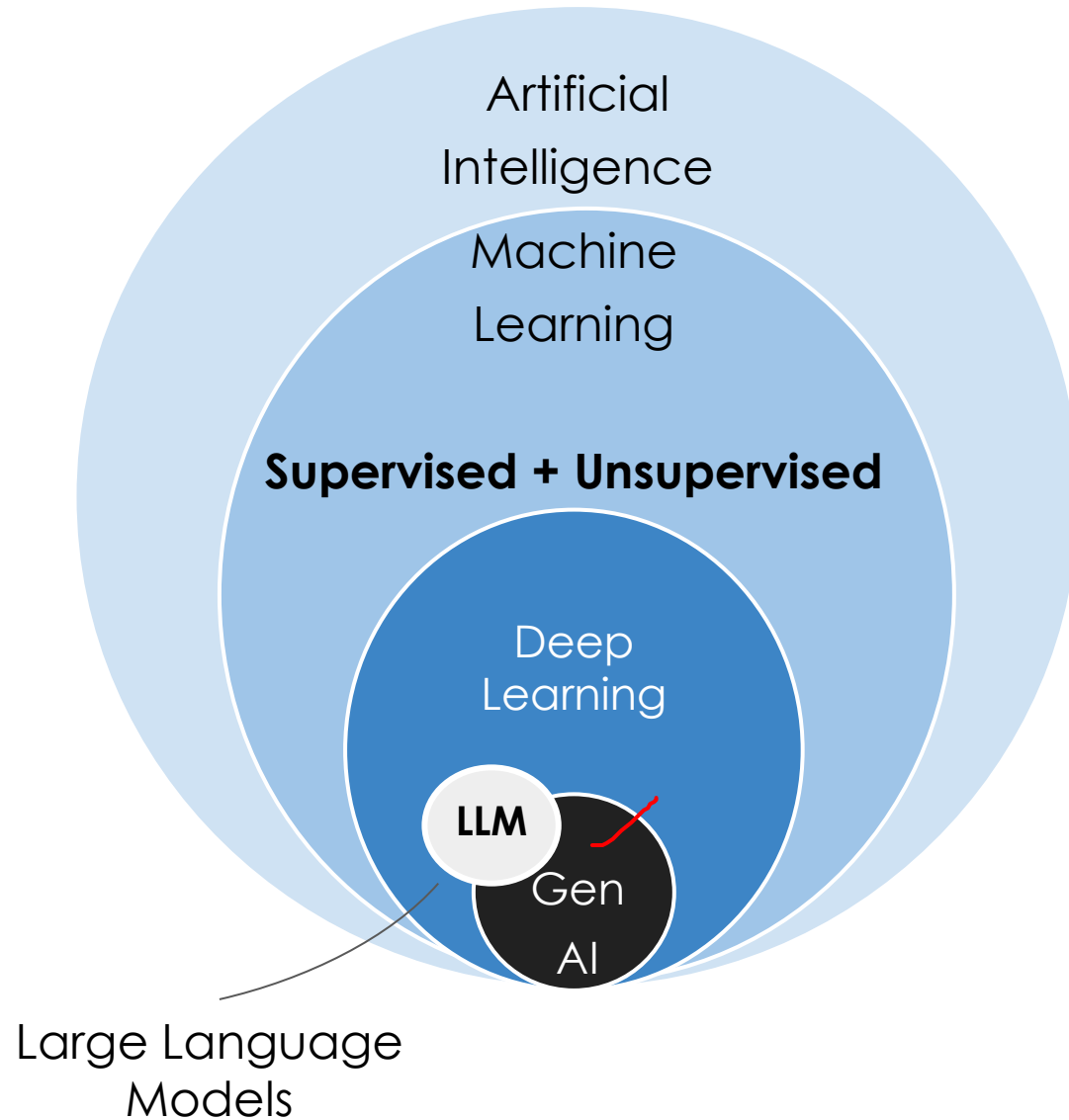
1990's

2000's

2010s

ORACLE®

Copyright © 2018, Oracle and/or its affiliates. All rights reserved. |



Brief history

14y

CNN



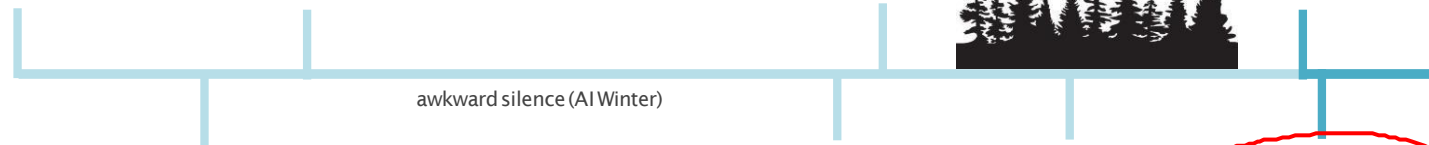
1958 Perceptron

1974 Backpropagation

1998 Convolution Neural Networks for Handwritten Recognition



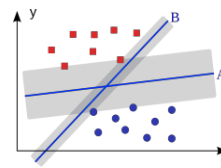
Google Brain Project 16k Cores 2012



1969 Perceptron criticized



1995 SVM reigns



2006 Restricted Boltzmann Machine



2012 AlexNet wins ImageNet

IMAGENET

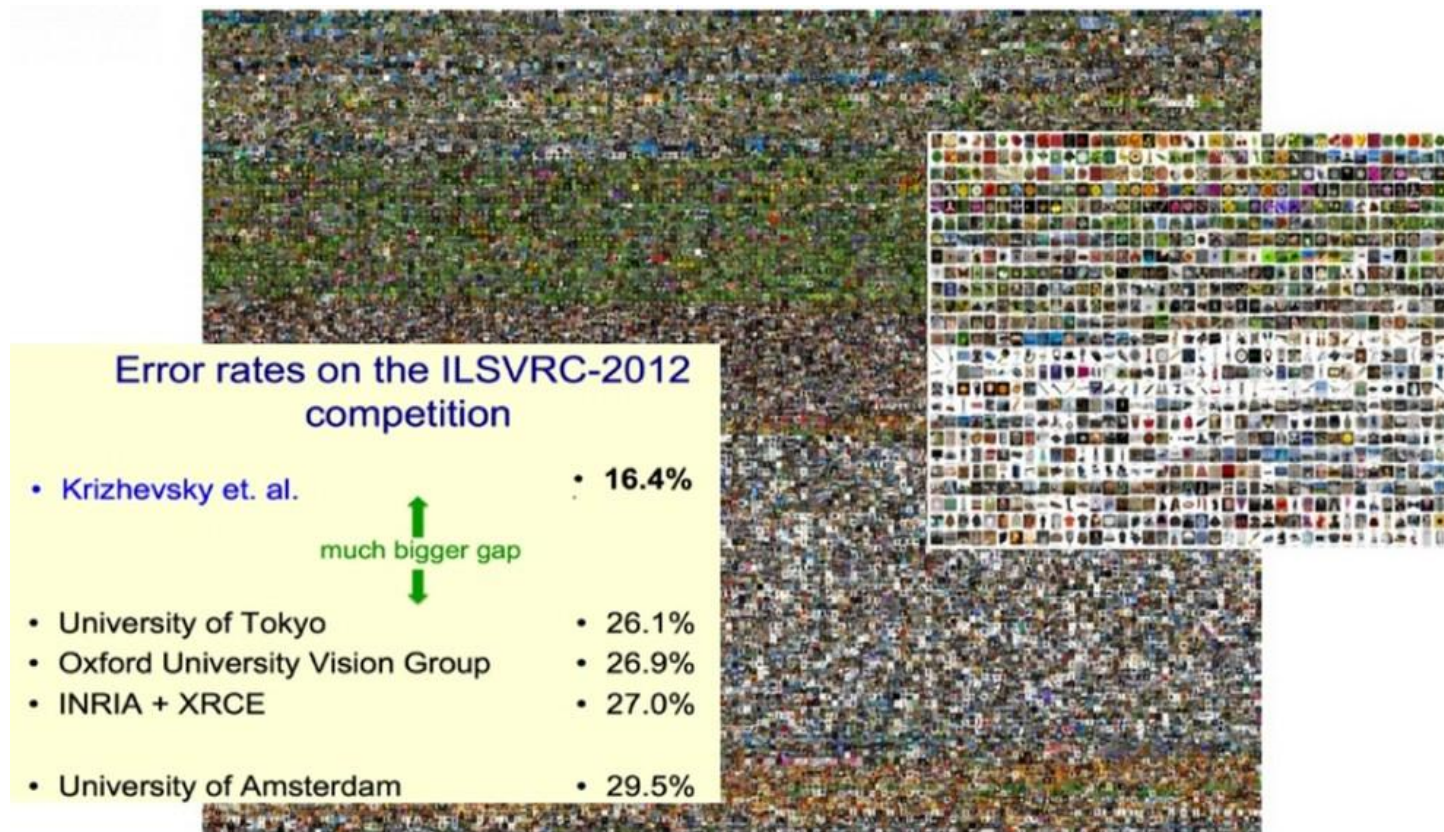
G.H

Ref:-

<https://www.slideshare.net/mohamedloey/deep-learning-71352250>

ImageNet Challenge 2012

1.2M images with 1000 object categories



Why second wave?

- More data from systems and sensors (IoT)
- More compute power : GPU's, multi-core CPU's
- Can train deep architectures
- **Some more applications** of DL are:
- Image classification, natural language processing, chat bots, personalized recommendations, prediction, anomaly detection, fraud detection, drug discovery, autonomous cars, video analytics etc...

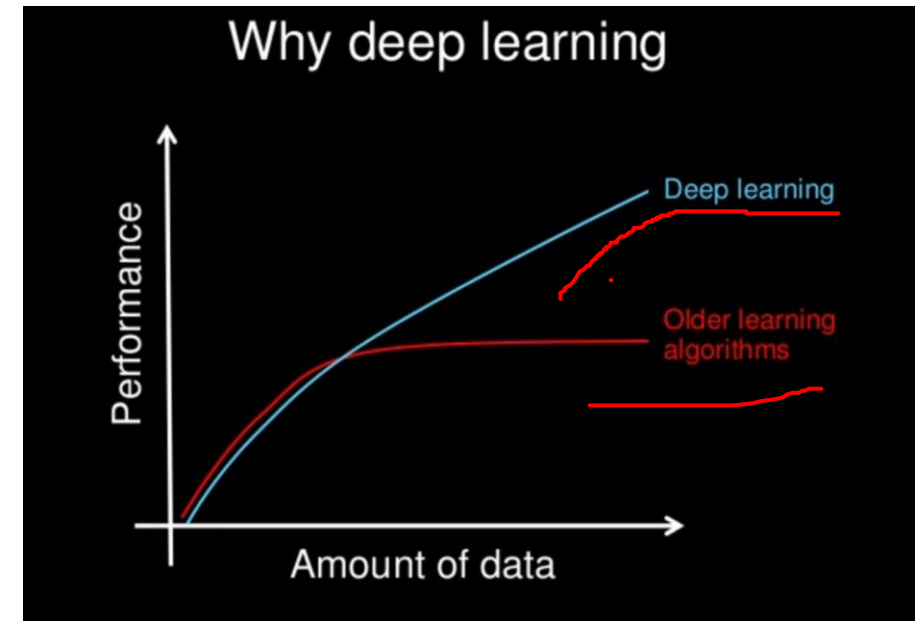
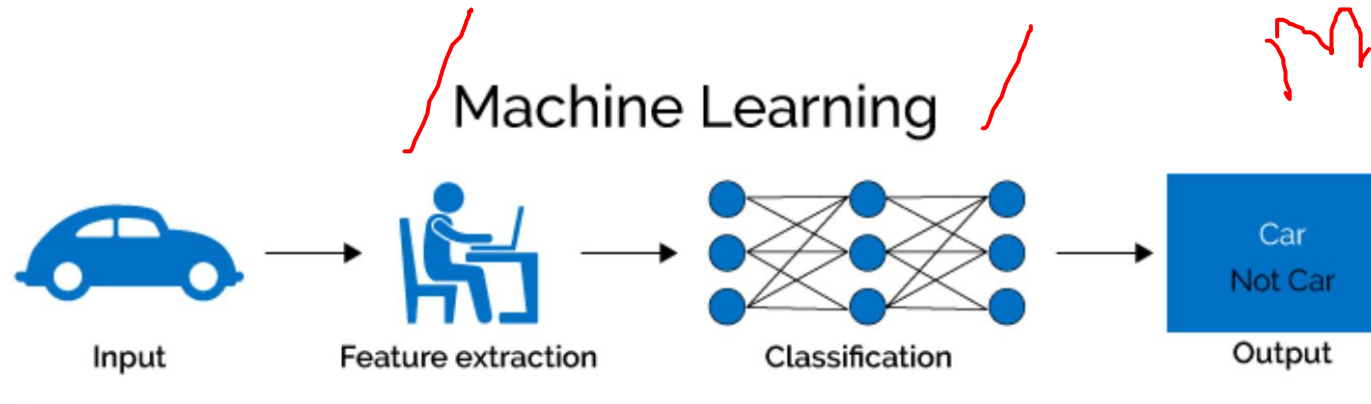


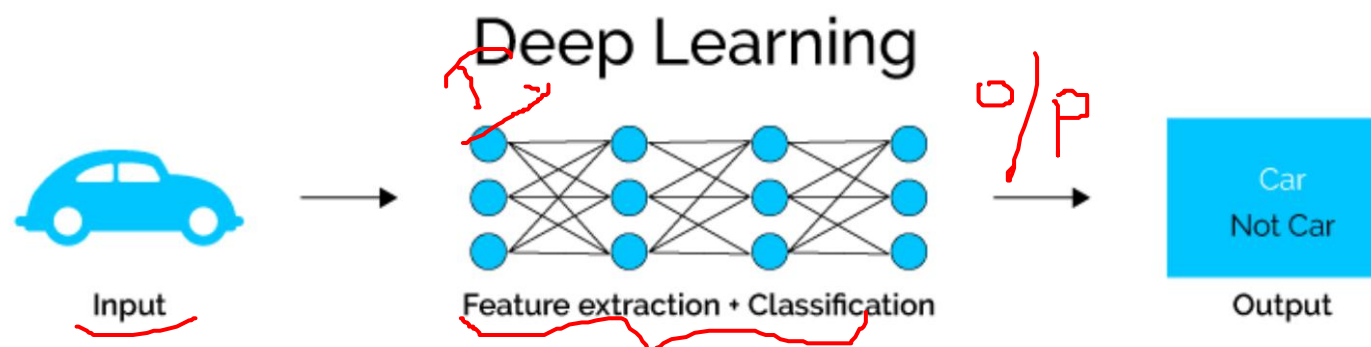
Fig Ref:- <https://www.linkedin.com/pulse/how-artificial-intelligence-revolutionizing-finance-del-toro-barba/>

Machine learning vs Deep learning

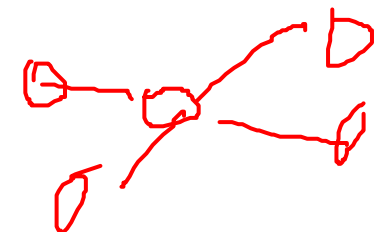
351



ML

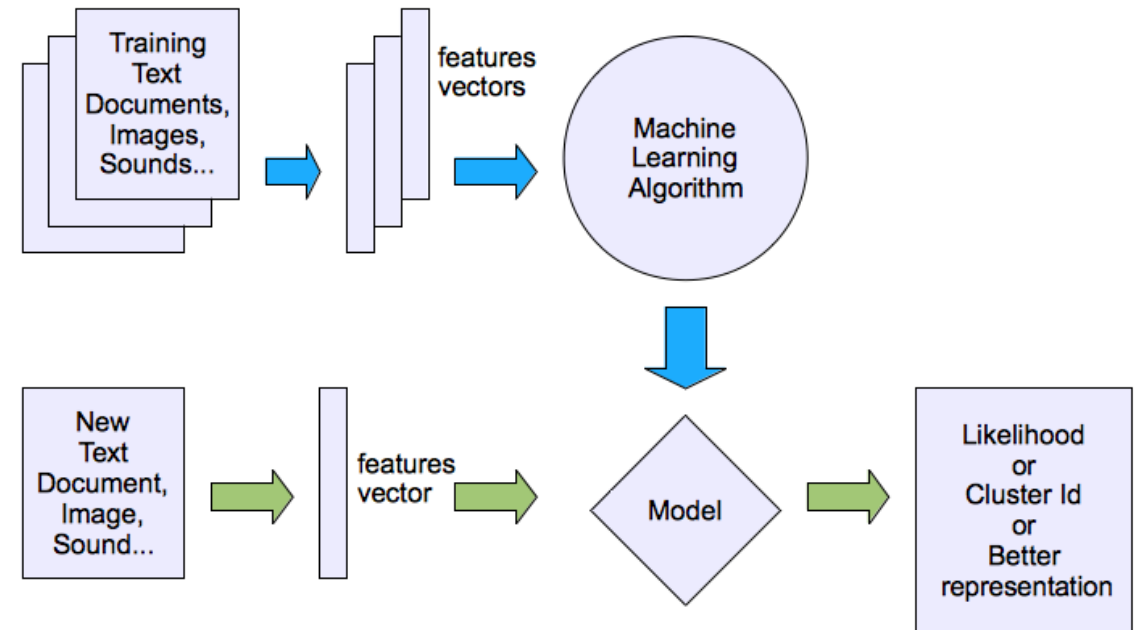
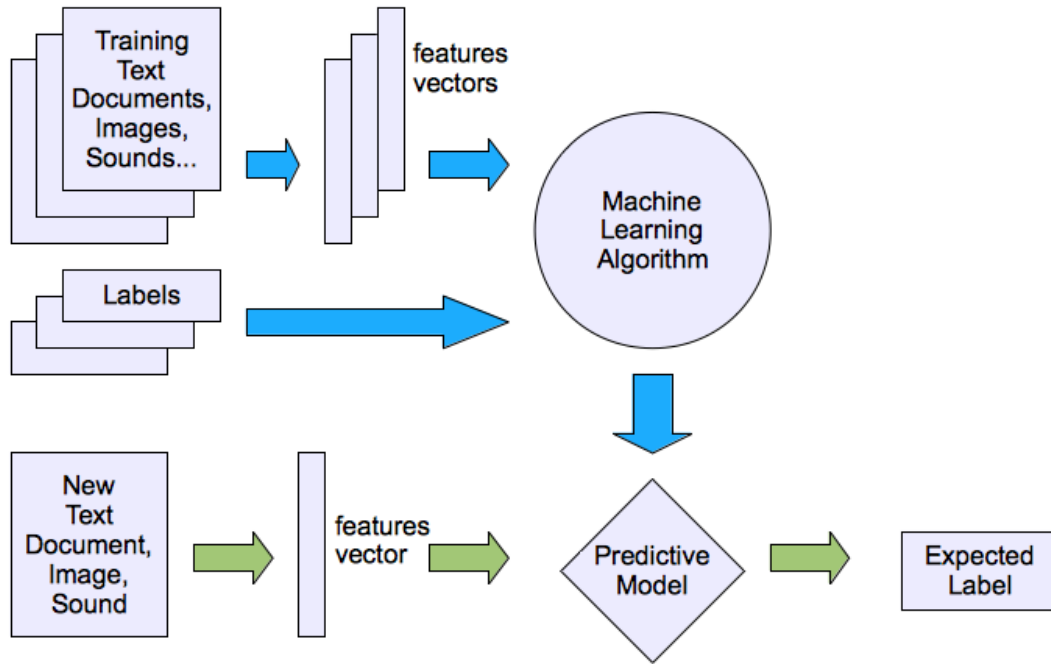


O/P

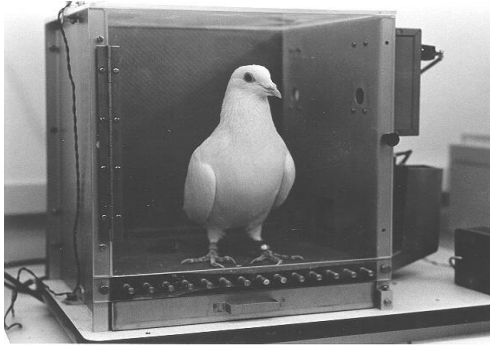


Ref:- <https://www.xenonstack.com/blog/log-analytics-with-deep-learning-and-machine-learning>

Supervised & unsupervised learning

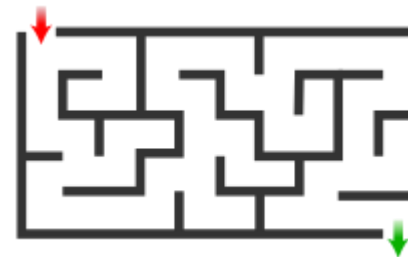


Thinking is possible even with a “small” brain



Pigeons as art experts – 85% (Watanabe *et al.* 1995)

Source: https://en.wikipedia.org/wiki/Marc_Chagall
https://en.wikipedia.org/wiki/Vincent_van_Gogh



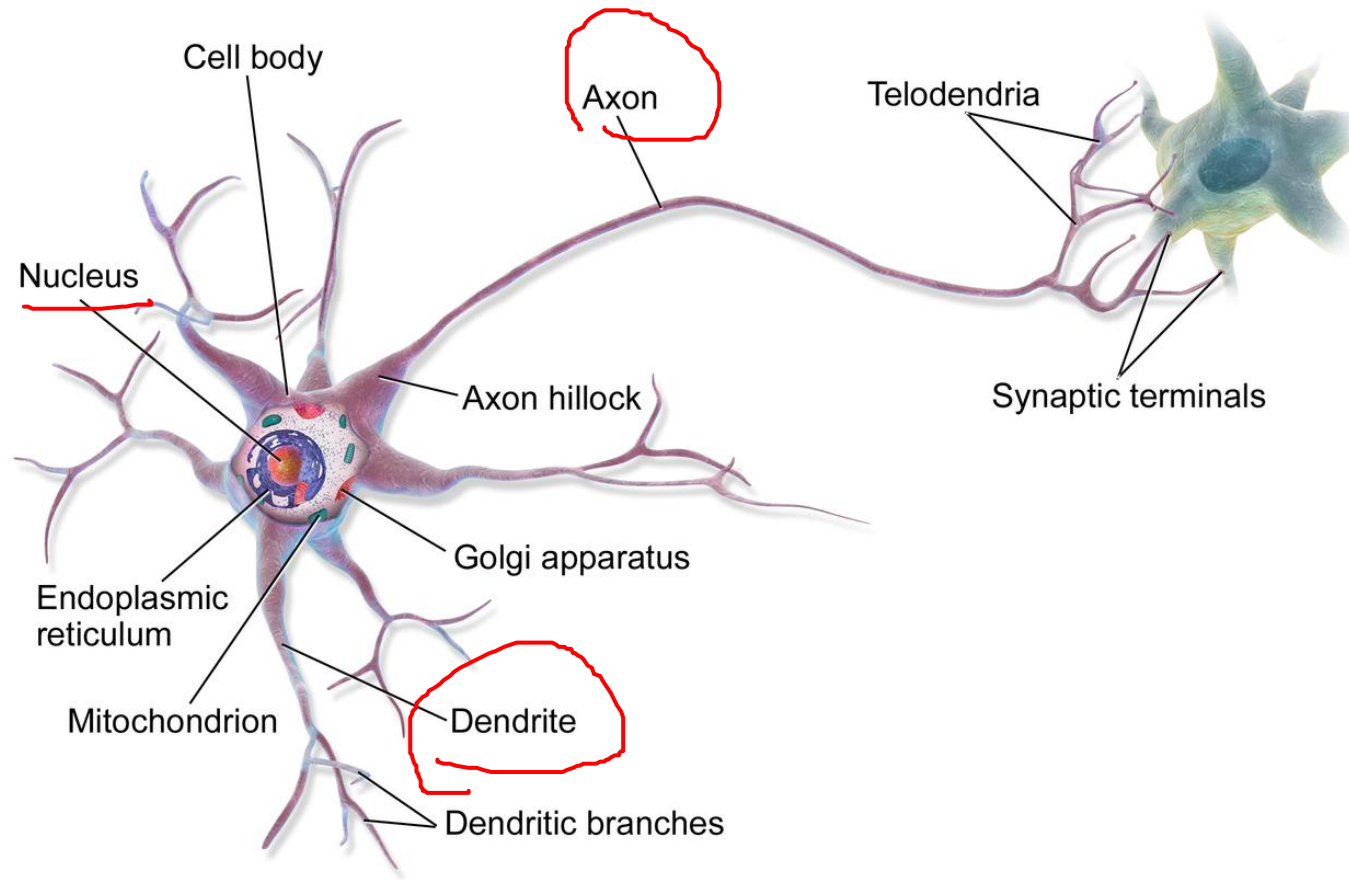
Mice trained to run mazes[1], detect drugs[1][2]



The results

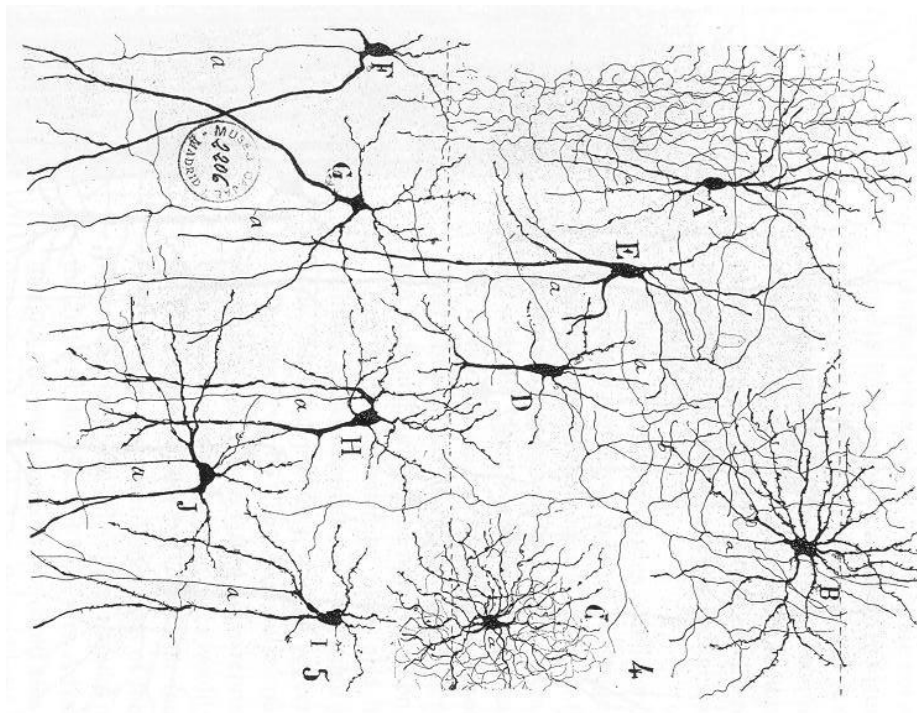
- Pigeons were able to discriminate between Van Gogh and Chagall with 95% accuracy (when presented with pictures they had been trained on)
- Discrimination still 85% successful for previously unseen paintings of the artists
- Mice can memorize mazes, odors of contraband (drugs / chemicals / explosives)

So, how does the brain work?



Direction of signal is along the Axon from Nucleus to synapse

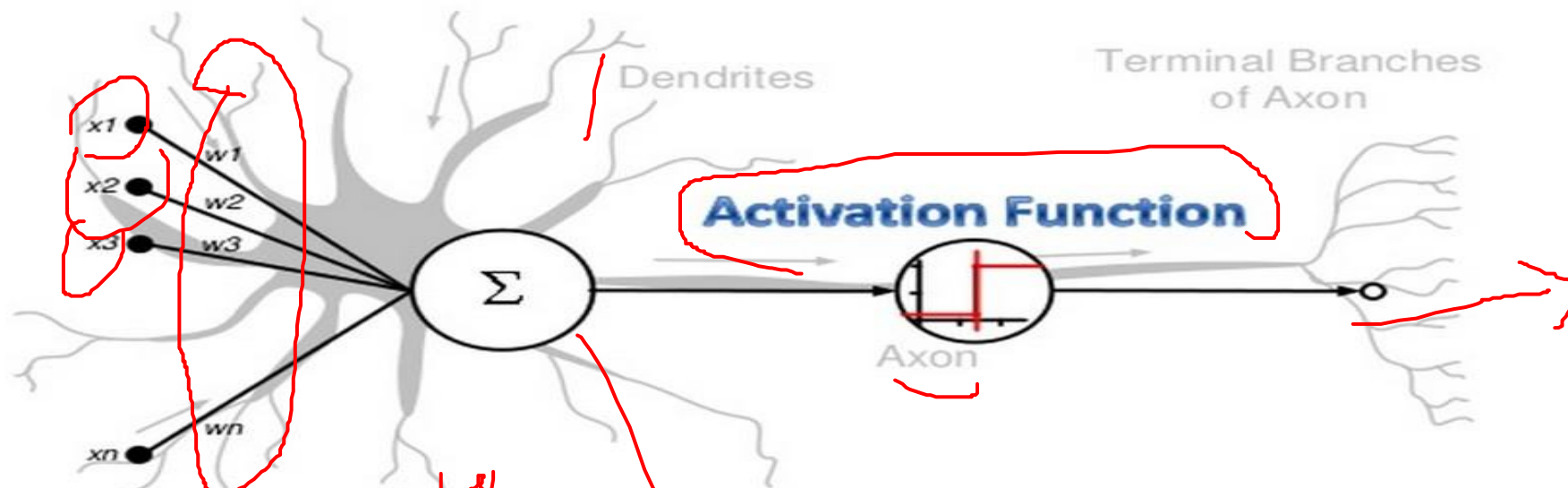
Biological neural networks



- Fundamental units are termed neurons.
- Connections between neurons are synapses.
- Adult human brain consists of 100 billion neurons and 1000 trillion synaptic connections.
- Equivalent to computer with one trillion bit per second processor.

Artificial neural network

$$\sigma(e^{xL+w} + b)$$

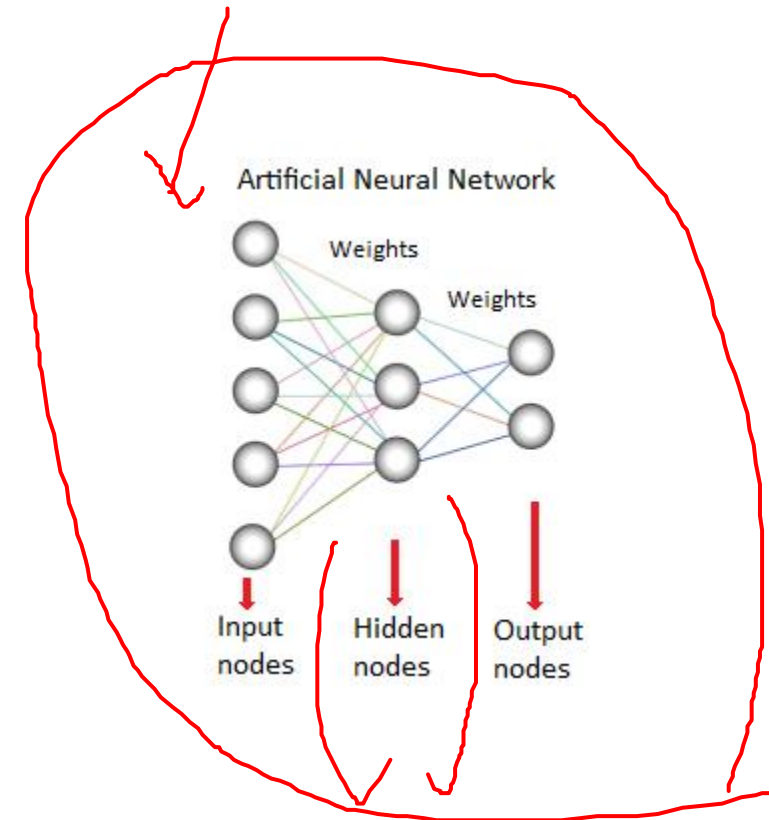


$$e^{xL/w} + \text{weights}$$

Slide credit : Andrew L. Nelson

Fundamentals

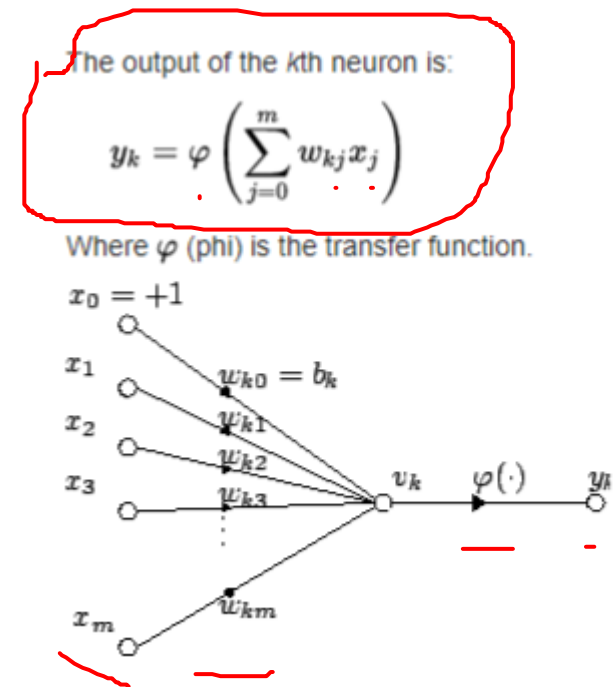
- DL is a class of ML algorithms use a cascade of many layers of **nonlinear processing** units for **feature extraction** and **transformation**.
- A neural net model is composed a set of **Layers**.
- There are many types of layers available and each layer has many parameters. Thus we can have infinitely many different network architectures.
- **Tensor flow demo** with neural network



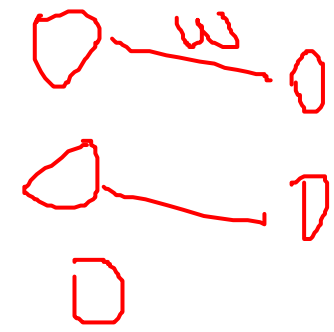
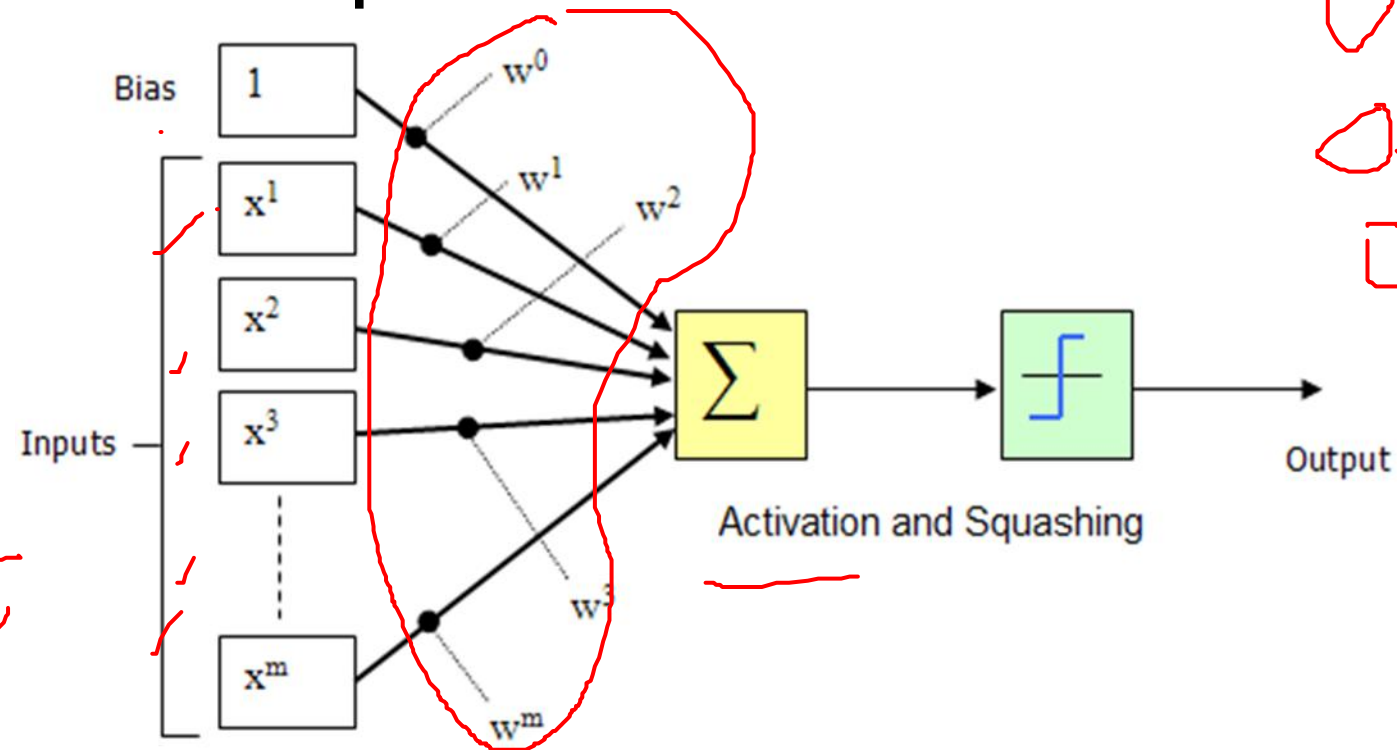
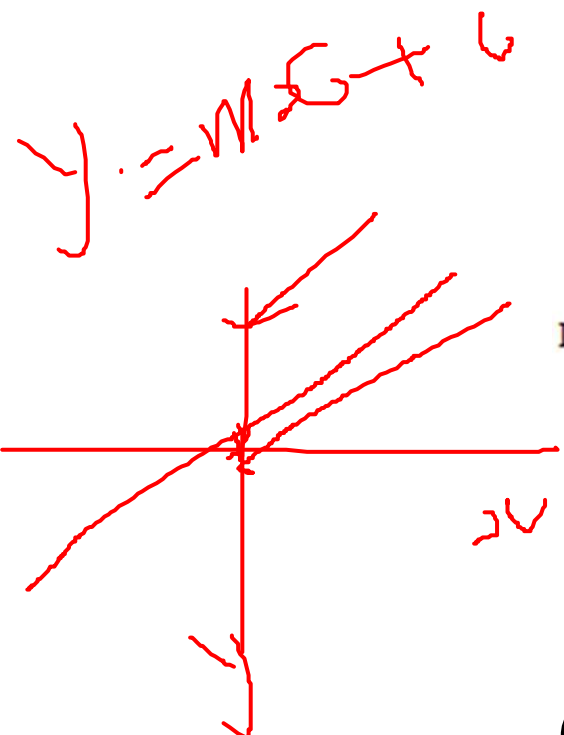
Artificial neural network

Artificial **neurons** are elementary units in an artificial neural network. The artificial neuron receives one or more **inputs** (representing dendrites) and sums them to produce an **output** (or activation) (representing a neuron's axon).

Usually the sums of each node are weighted, and the sum is passed through a non-linear function known as an **activation** function.



Artificial neural model: Perceptron



$$\text{Output} = \{W_0 + X_1W_1 + X_2W_2 + X_3W_3 + \dots > T\}$$

$$Y = w X + b$$

$-1 < Y < 1$
Equation looks similar to some other model? (Linear/Logistic regression)

Learning process

These equations

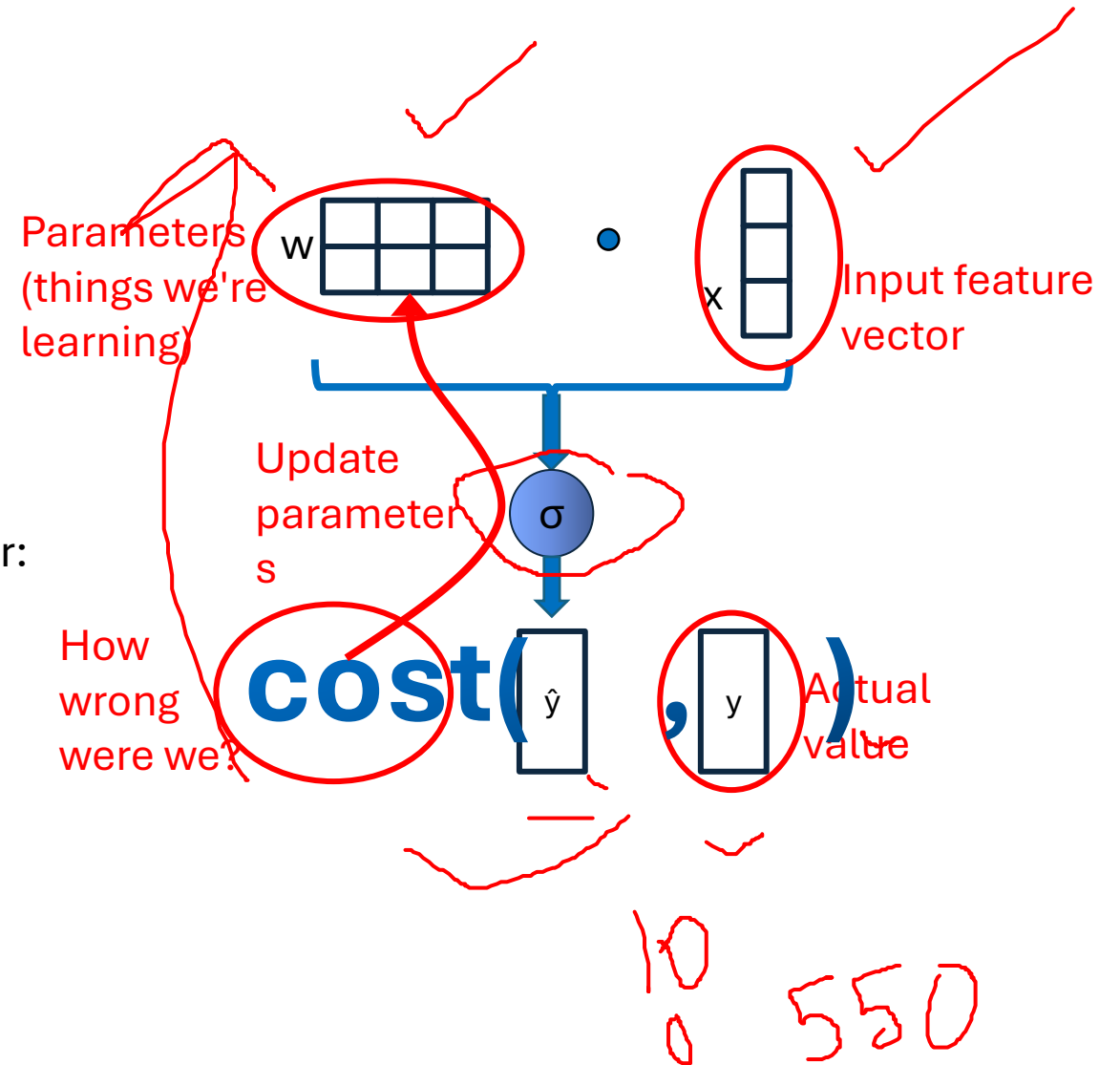
$$3x + 2y + 5z = 7$$

$$1x + 3y + 9z = 2$$

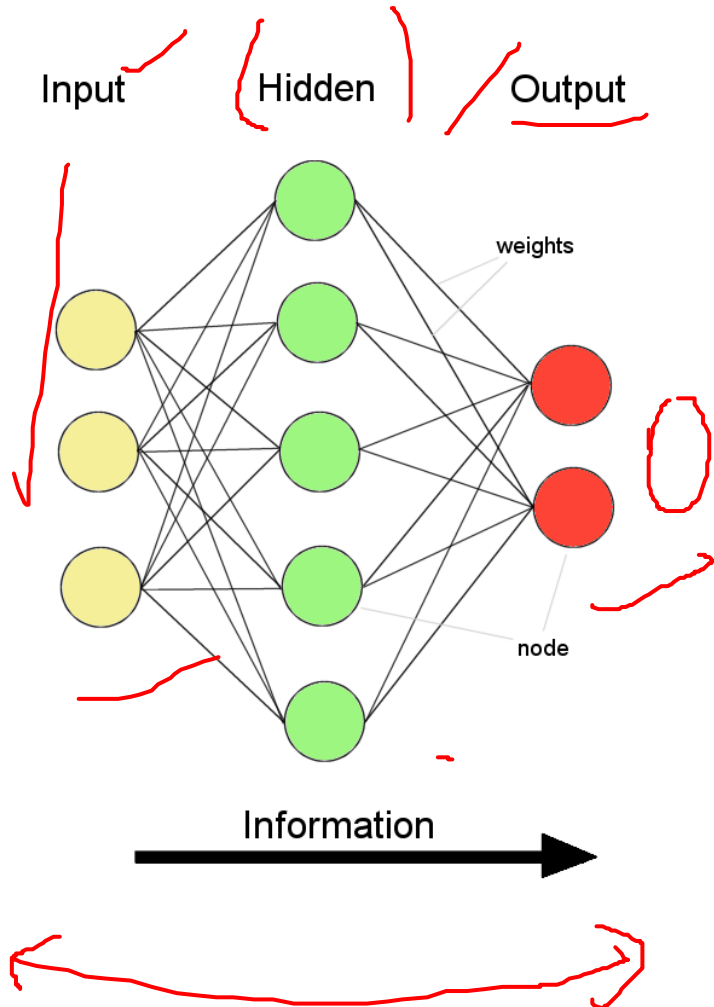
$$4x + y + 0z = 8$$

can be written as a dot product of a matrix and a vector:

$$\begin{bmatrix} 3 & 2 & 5 \\ 1 & 3 & 9 \\ 4 & \frac{1}{2} & 0 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 7 \\ 2 \\ 8 \end{bmatrix}$$



Feed forward nets



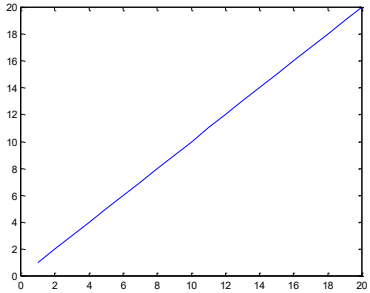
Information flow is unidirectional

- Data is presented to *Input layer*
 - Passed on to *Hidden Layer*
 - Passed on to *Output layer*
- Information is distributed
 - Information processing is parallel
 - Backpropagation
- Requires training set (input / output pairs)
 - Starts with small random weights
 - Error is used to adjust weights (supervised learning)

What are hidden layers?

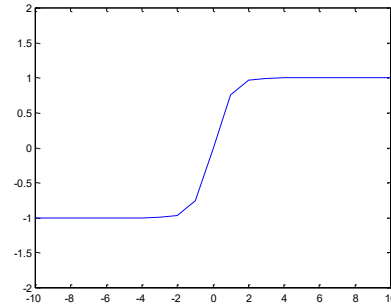
- They are non-linear sums of the inputs
- So, they are not linearly dependent on inputs (they are non-linearly dependent)
- They are engineered features of the original inputs!
 - Feature engineering is so difficult because for each type of data and each type of problem, different features do well
 - Neural networks can potentially **build features hierarchically**

Activation functions



Linear

$$y = x$$



Rectifier / ramp function

$$f(x) = \max(0, x)$$

x is the input to a neuron.

smooth approximation to the rectifier

is softplus

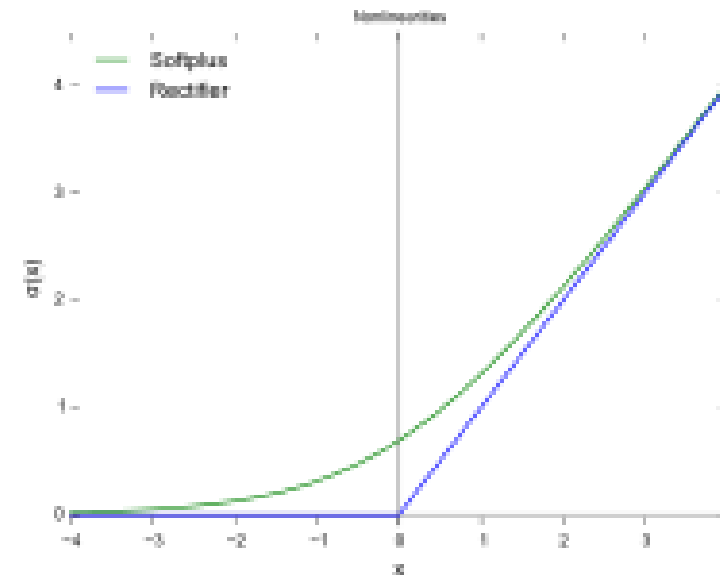
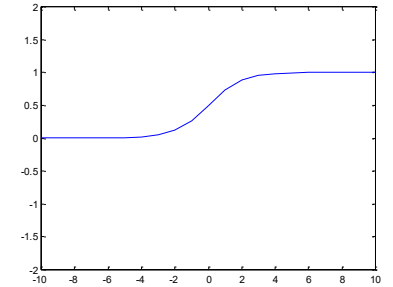
$$f(x) = \ln(1 + e^x)$$

Hyperbolic tangent

$$y = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$

Logistic

$$y = \frac{1}{1 + \exp(-x)}$$



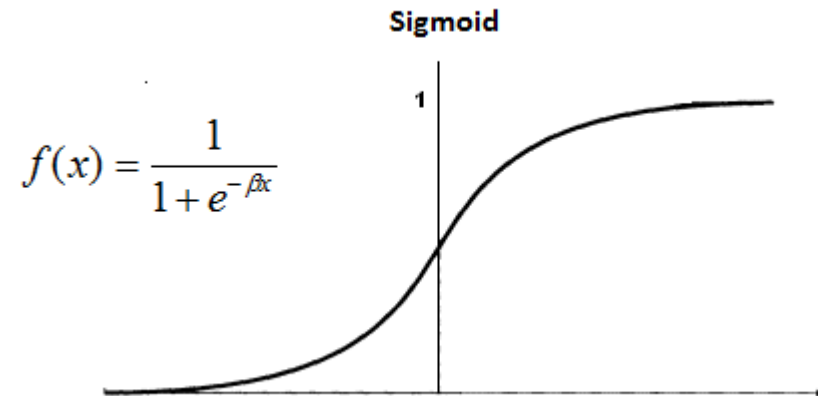
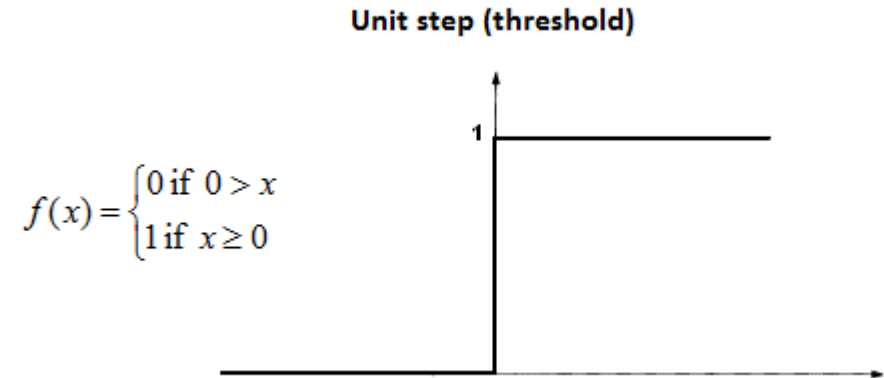
Activation functions

Unit step (threshold):

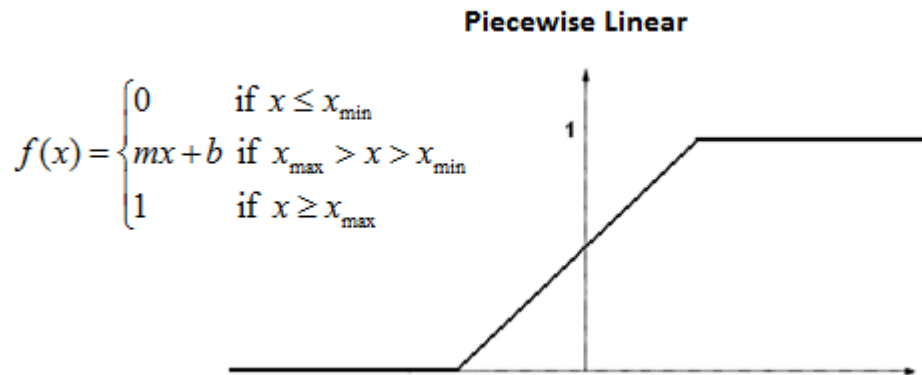
The transfer function translates the input signals to output signals. Four types of transfer functions are commonly used, Unit step (threshold), sigmoid, piecewise linear, and Gaussian.

Sigmoid:

The sigmoid function consists of 2 functions, logistic and tangential. The values of logistic function range from 0 and 1 and -1 to +1 for tangential function.

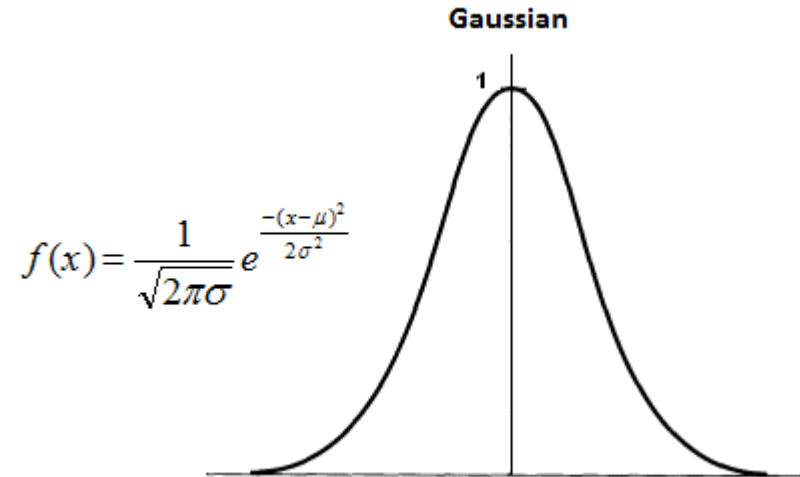


Activation functions



Piecewise Linear

The output is proportional to the total weighted output.



Gaussian

Gaussian functions are bell-shaped curves that are continuous. The node output (high/low) is interpreted in terms of class membership (1/0), depending on how close the net input is to a chosen value of average.

Properties of activation function

- **Nonlinear:**
 - When the activation function is non-linear, then a two-layer neural network can be proven to be a universal function approximator.
- **Continuously differentiable:**
 - This property is necessary for enabling gradient-based optimization methods.
 - The binary step activation function is not differentiable at 0, and it differentiates to 0 for all other values, so gradient-based methods can make no progress with it.
- **Range:**
 - When the range of the activation function is finite, gradient-based training methods tend to be more stable, because pattern presentations significantly affect only limited weights.
 - When the range is infinite, training is generally more efficient because pattern presentations significantly affect most of the weights. Smaller learning rates are typically necessary.
- https://en.wikipedia.org/wiki/Activation_function

Properties of activation function

- **Monotonic:**

- When the activation function is monotonic, the error surface associated with a single-layer model is guaranteed to be convex.

- **Smooth**

- Functions with a Monotonic derivative have been shown to generalize better in some cases.
- The argument for these properties suggests that such activation functions are more consistent with Occam's razor.

- **Approximates identity near the origin:**

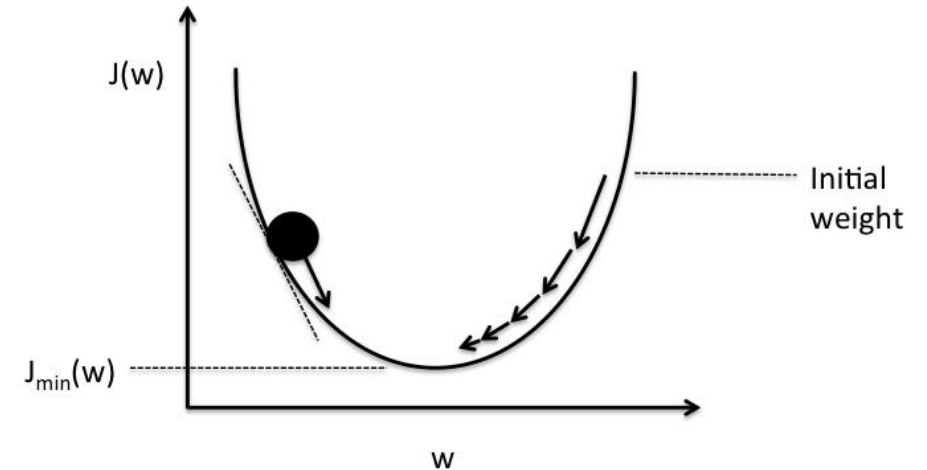
- When activation functions have this property, the neural network will learn efficiently when its weights are initialized with small random values.
- When the activation function does not approximate identity near the origin, special care must be used when initializing the weights.

But, how to learn hidden weights

- We need $\frac{\partial E}{\partial w}$. But, what is the error in internal nodes?
- Back propagation is the answer

Back propagation

- With the concept of **gradient descent**
 - Forward propagation: Activation [input * weight matrix]
 - To get optimal value of each weight:
 - Direction is opp. to the gradient
 - Find weight with minimum error
 - Derivative (slope of a tangent line - rate of change of a function)
 - Partial derivative (wrt one of the variables)
 - Chain rule (derivatives of composite functions)
 - calculate the error wrt each weight
 - **New weight = old weight - Derivative Rate * learning rate**



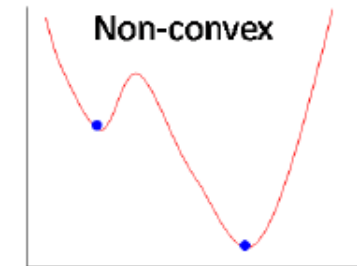
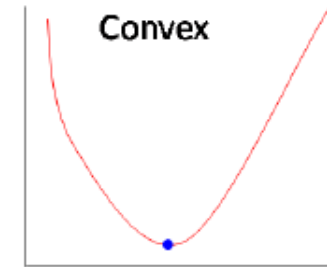
Schematic of gradient descent.

Fig Ref:-

<http://datathings.com/blog/post/neuralnet/>

Gradient descent

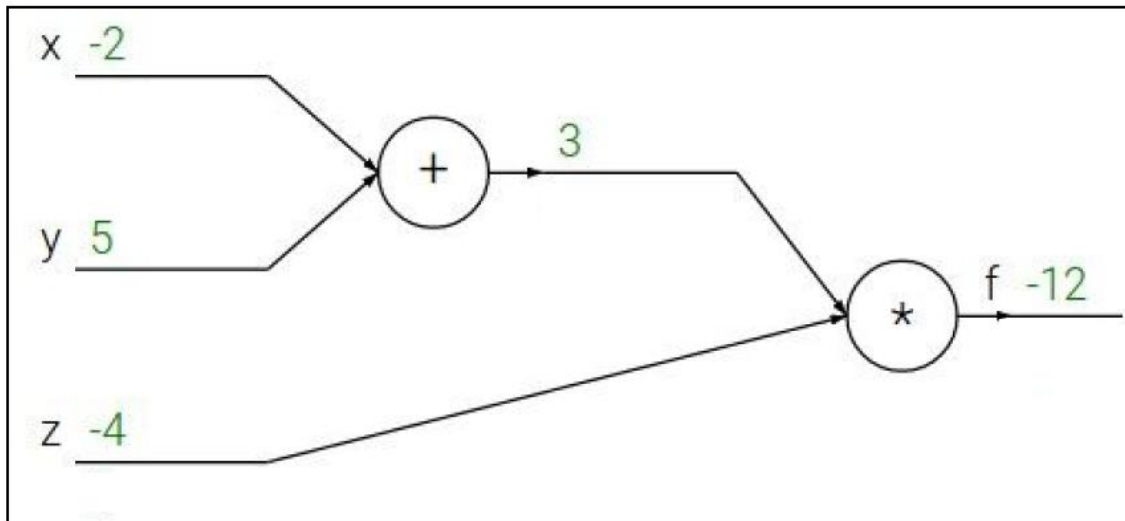
- Randomly start somewhere and compute slope using calculus. Update the coefficients in the negative slope direction
- Repeat until convergence
- Gradient descent is greedy
- Multiple initiations and selection of the best is the option
- No way to theoretically guarantee convergence to global optimum.
- But, generally convergence to a local optimum which is very close to global.
- Therefore state-of-the-art results :)



Back propagation algorithm

There were no training algorithms for multi layered networks until 80s. Geoff Hinton, then a young professor at Carnegie-Mellon University, solved it with back-propagation algorithms. He used chain rule of differentiation to propagate the error backwards and update the weights.

$$f(x, y, z) = (x + y)z$$

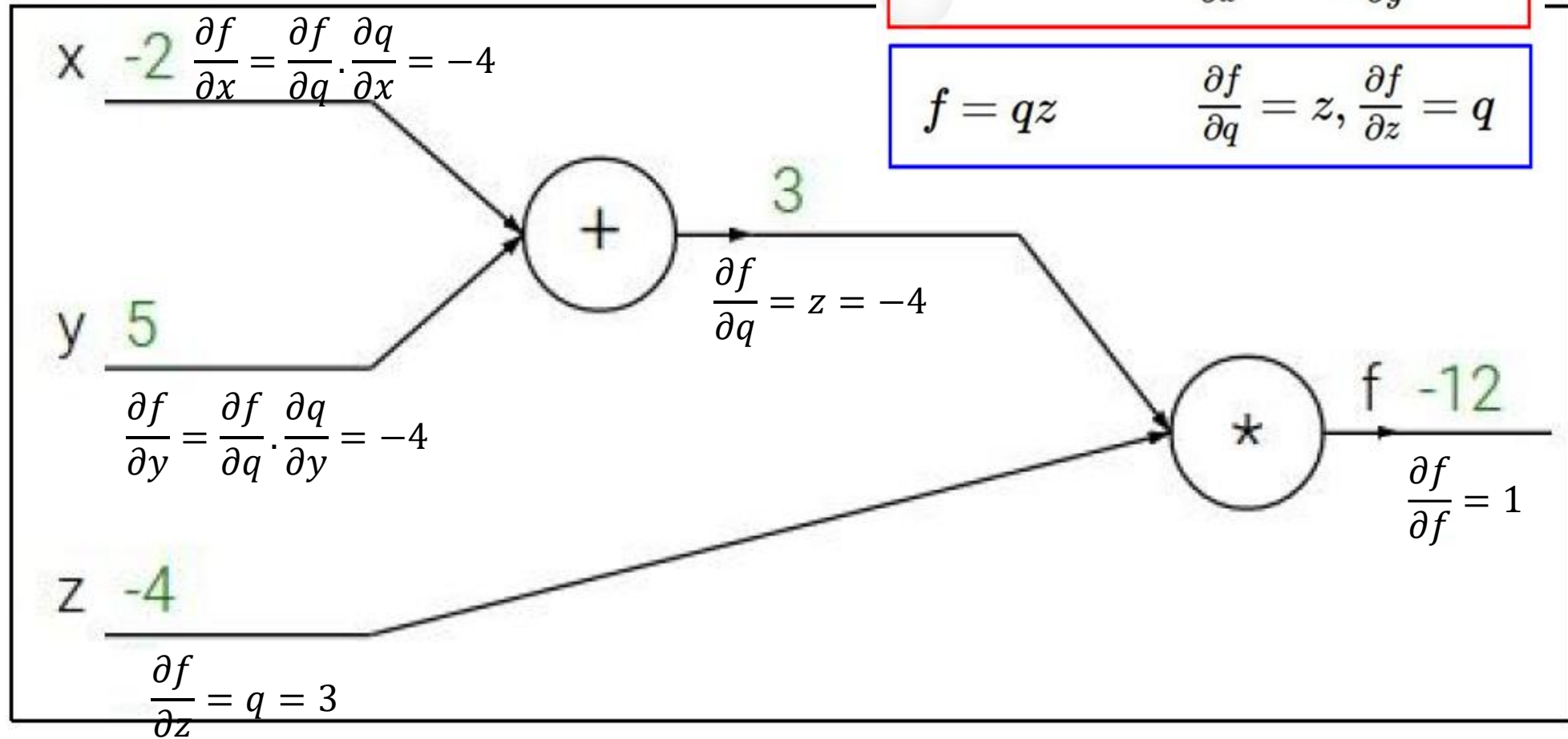


$$x = -2, y = 5, z = -4$$

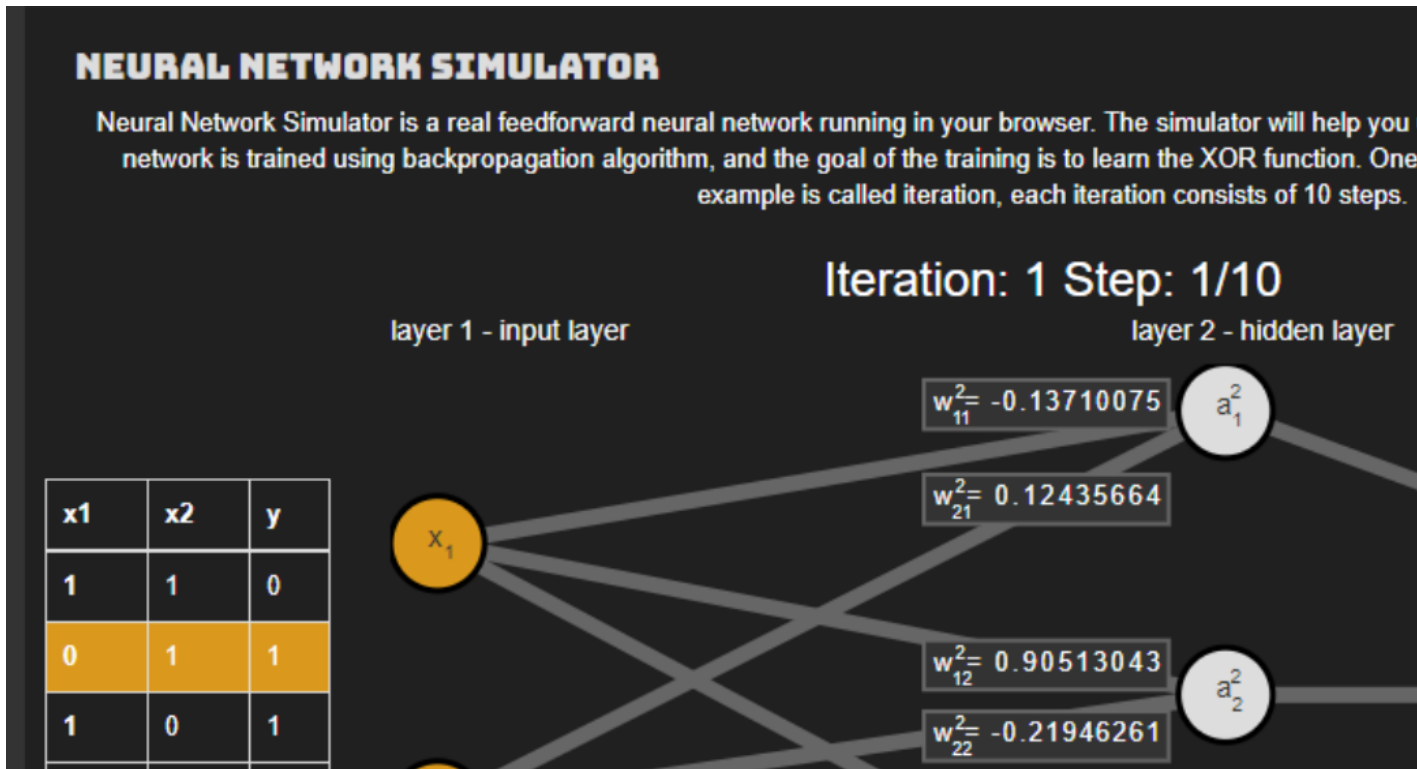
$$\text{Want: } \frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$



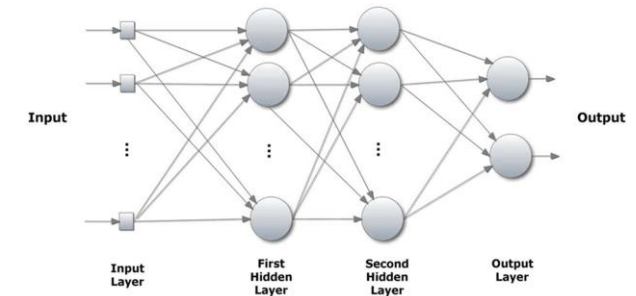
Neural Network simulator



- www.mladdict.com/neural-network-simulator
- <https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/>

Artificial neural networks

Output of one layer is input to another.



Layer1:
$$h_1 = f_1(x) = \sigma(w_1x + b_1)$$

Layer2:
$$h_2 = f_2(h_1) = \sigma(w_2h_1 + b_2)$$

Layer3:
$$h_3 = f_3(h_2) = \sigma(w_3h_2 + b_3)$$

.

.

.

LayerN:
$$\text{output} = f_N(h_{N-1}) = \sigma(w_Nh_{N-1} + b_N)$$

$$\text{output} = f_N(f_{N-1}(f_{N-2}(\dots(f_1(x)))) = f(x)$$

Avoid Over Fitting

- Dropouts
- Early stopping
- Use Data Augmentation
- Use regularization

Thank you

Happy Learning